CSE 4165 FINAL EXAM MASTER CHEAT SHEET • SIDE 1: ALL TYPES OF Q1 (JS) & Q2 (PHP)

Time: 1 hr 55 min | 30 Marks | Time Budget: Q1 30m • Q2 35m • Q3 40m • Zip & Upload 10m

A4 DUPLEX • SIDE 1

Q1 UNIVERSAL HTML SKELETON (NO CSS NEEDED)

HTML SKELETON

```
<!— index.html: 1 or 2 inputs + button + label placed below —>
<input type="text" id="inputA" placeholder="Name / First Input">
<input type="number" id="inputB" step="any" placeholder="Value / Numbe
r">
<button id="actionBtn">Add / Calculate</button>
<label id="outLabel"></label>
<script src="script.js"></script>
```

TYPE 1A: RUNNING TOTAL & GOAL PROGRESS TRACKER (Calorie, Study, Expense)

TYPE 1A

```
const GOAL = 2000; // Target from exam (e.g. 2000 kcal, 20 hrs, 100 pt
s)
let total = 0, count = 0;
const historyList = []; // Stores entered subjects/meals

function handleAction() {
  const val = parseFloat(document.getElementById("inputB").value);
  const name = document.getElementById("inputA").value.trim();
  if (isNaN(val) || val ≤ 0) { alert("Enter valid positive number"); r
eturn; }
  count++; total += val;
  if (name) historyList.push(name);
  const avg = total / count;

  // 4-Tier Ladder (Order: smallest first with ≤ OR largest first with
≥)
  let msg = "";
  if (total ≤ 800)       msg = "You're off to a healthy start!";
  else if (total ≤ 1600) msg = "Good progress, keep it balanced!";
  else if (total < GOAL)  msg = "Almost at your limit!";
  else                    msg = "Goal reached! Stay mindful!";

  // Exam Warning Limit: Too many entries/sessions without reaching goa
l
  if (count > 10 && total < GOAL) msg += " Be cautious of frequent snac
king!";

  document.getElementById("outLabel").textContent =
    `Total: ${total} | Avg: ${parseFloat(avg.toFixed(2))} | Items: [${h
istoryList.join(", ")}] | ${msg}`;
  document.getElementById("inputB").value = "";
}
document.getElementById("actionBtn").addEventListener("click", handleAc
tion);
```

TYPE 1B: CRITERIA & POINTS SCORER (Password Strength, Scoring)

TYPE 1B

```
let attempts = 0;
function checkScore() {
  const p = document.getElementById("inputA").value;
  attempts++;
  let score = 0;
  // Criteria 1: Length points (e.g. +10 per 2 chars above min 6 chars)
  if (p.length ≥ 6) score += Math.floor((p.length - 6) / 2) * 10;
  // Criteria 2-5: Regex tests for character sets
  if (/[A-Z]/.test(p))      score += 15; // Uppercase
  if (/[a-z]/.test(p))      score += 15; // Lowercase
  if (/[0-9]/.test(p))      score += 20; // Number
  if (/[!@#$%^&*]/.test(p)) score += 25; // Special char

  // Tiers: Very Weak (0-30), Weak (31-50), Medium (51-70), Strong (71-
90), Very Strong (91+)
  let lvl = (score ≤ 30) ? "Very Weak" : (score ≤ 50) ? "Weak" :
            (score ≤ 70) ? "Medium" : (score ≤ 90) ? "Strong" : "Very
Strong";
  let out = (score ≥ 100) ? "Perfect Password! Success!" : `Score: ${s
core} - ${lvl}`;

  // Warning on > N attempts without reaching "Strong" (score < 71)
  if (attempts > 8 && score < 71) out += " | Need practice! Tips: Add u
ppercase, digits & symbols.";
  document.getElementById("outLabel").textContent = out;
}
```

TYPE 1C: MULTI-INPUT SENSOR & FORMULA EVALUATOR (Patient Vitals)

TYPE 1C

```
const listA = [], listB = []; // Keeps track of all readings
function computeFormula() {
  const vA = parseFloat(document.getElementById("inputA").value);
  const vB = parseFloat(document.getElementById("inputB").value);
  if (isNaN(vA) || isNaN(vB)) return;
  listA.push(vA); listB.push(vB);

  // Averages using reduce
  const avgA = listA.reduce((s, n) => s + n, 0) / listA.length;
  const avgB = listB.reduce((s, n) => s + n, 0) / listB.length;

  // Formula with absolute difference: Math.abs(vA - 80)
  const score = (100 - vB) * 2 + Math.abs(vA - 80) * 0.5;
  const status = (score ≤ 10) ? "SAFE" : (score ≤ 20) ? "WARNING" :
"DANGER";

  document.getElementById("outLabel").textContent =
    `Avg A: ${avgA.toFixed(1)} | Avg B: ${avgB.toFixed(1)} | Score: ${s
core.toFixed(1)} | Status: ${status}`;
}
```

Q2 UNIVERSAL HTML FORM (EXACTLY 3 INPUTS)

FORM SKELETON

```
<!— form.html: form action MUST target calculate.php with method POST
—>
<form action="calculate.php" method="POST">
  <label>Input 1: <input type="number" name="val1" step="any" required>
</label><br>
  <label>Input 2: <input type="number" name="val2" step="any" required>
</label><br>
  <label>Input 3: <input type="number" name="val3" step="any" required>
</label><br>
  <input type="submit" value="Calculate">
</form>
```

TYPE 2A: RESOURCE CEILING ALLOCATION & WASTE (Venues, Screens, Batteries)

TYPE 2A

```
<?php // calculate.php - Type 2A
function calculateBooking($people, $costPerPerson, $capacity) {
  // 1. Cannot book partial units: always round UP with ceil()
  $unitsNeeded = ceil($people / $capacity);
  // 2. Empty / Unused capacity
  $totalCapacity = $unitsNeeded * $capacity;
  $emptyUnits = $totalCapacity - $people;
  // 3. Wasted money: empty seats * ticket/person price
  $wastedMoney = $emptyUnits * $costPerPerson;

  // 4. Efficiency status (% unused of total carried)
  $unusedPct = ($totalCapacity > 0) ? ($emptyUnits / $totalCapacity) *
100 : 0;
  $status = ($unusedPct ≤ 10) ? "Efficient" : (($unusedPct ≤ 25) ? "A
cceptable" : "Wasteful");

  return ['units' => $unitsNeeded, 'empty' => $emptyUnits, 'waste' =>
$wastedMoney, 'status' => $status];
}
if ($_SERVER["REQUEST_METHOD"] == "POST") {
  $v1 = intval($_POST['val1']); $v2 = intval($_POST['val2']); $v3 = int
val($_POST['val3']);
  $r = calculateBooking($v1, $v2, $v3);
  echo "<table border='1' cellpadding='6'>
        <tr><th>People</th><th>Rate</th><th>Cap</th><th>Units</th><th>E
mpty</th><th>Wasted BDT</th><th>Status</th></tr>
        <tr><td>$v1</td><td>$v2</td><td>$v3</td><td>{$r['units']}</td><
td>{$r['empty']}</td>
            <td>" . number_format($r['waste']) . "</td><td>{$r['statu
s']}</td></tr></table>";
} ?>
```

TYPE 2B: PRODUCTION VS TARGET DIFFERENCE (Event Sales, Credits)

TYPE 2B

```
<?php // calculate.php - Type 2B
function evaluatePerformance($rate, $duration, $target) {
  $total = $rate * $duration; // e.g. items/day * days OR credits/cours
e * courses
  // Classification
  $grade = ($total ≥ 500) ? "Excellent" : (($total ≥ 300) ? "Good" :
(($total ≥ 150) ? "Average" : "Poor"));
  // Target difference exact matching
  $diff = $total - $target;
  if ($diff == 0)      $msg = "Target met exactly (0 difference)";
  elseif ($diff > 0)  $msg = "Above target by $diff";
  else                $msg = "Below target by " . abs($diff);

  return ['total' => $total, 'grade' => $grade, 'msg' => $msg];
} ?>
```

TYPE 2C: BEST 2 OF 3 MARKS & PASS/FAIL LOGIC

TYPE 2C

```
<?php // calculate.php - Type 2C
function processExamMarks($ct1, $ct2, $ct3, $mid, $final) {
  // Best two CTs: subtract the minimum from total of all three
  $bestTwoAvg = (($ct1 + $ct2 + $ct3) - min($ct1, $ct2, $ct3)) / 2.0;
  $totalMarks = $bestTwoAvg + $mid + $final;
  $status = ($totalMarks > 54) ? "Passed" : "Failed";
  return ['bestAvg' => $bestTwoAvg, 'total' => $totalMarks, 'status' =>
$status];
} ?>
```

CSE 4165 FINAL EXAM MASTER CHEAT SHEET • SIDE 2: PHP & MYSQL (Q3) & QUERY ENCYCLOPEDIA

Total Marks: 10M (database.sql: 5M + solution.php: 5M) | All 6 SQL Archetypes + Dictionary

A4 DUPLEX • SIDE 2

Q3 PART 1: database.sql SKELETON (ALWAYS DO FIRST) — DATABASE.SQL

```
CREATE DATABASE IF NOT EXISTS exam_db;
USE exam_db;
DROP TABLE IF EXISTS records;
CREATE TABLE records (
  id        INT PRIMARY KEY,
  name      VARCHAR(100),
  category  VARCHAR(50),
  price     DECIMAL(10,2), -- Money, Salary, Fees
  stock     INT,           -- Quantity, Attendees, Days
  rating    DECIMAL(3,1),  -- 4.5, 4.8
  status    VARCHAR(50)    -- 'Returned', 'Overdue', 'D'
);
INSERT INTO records (id, name, category, price, stock, rating, status)
VALUES
(1, 'Alpha Item', 'Tech', 450.00, 10, 4.8, 'Active'),
(2, 'Beta Item',  'Tech', 850.00,  5, 4.2, 'Active'),
(3, 'Gamma Item', 'Desk', 250.00, 20, 3.9, 'Pending'),
(4, 'Delta Item', 'Desk', 120.00, 15, 4.5, 'Overdue');
```

Tips: Double single-quotes for apostrophes: `'Cox''s Bazar'` • Dates format: `'2026-10-15'`

Q3 PART 2: solution.php FULL EXECUTION SCRIPT — SOLUTION.PHP

```
<?php
$conn = new mysqli("localhost", "root", "", "exam_db");
if ($conn->connect_error) die("Connection failed: " . $conn->connect_er
ror);

// 1. Reusable SELECT Table Printer (Dynamic Columns)
function showTable($conn, $title, $sql) {
  echo "<h4>$title</h4>";
  $res = $conn->query($sql);
  if (!$res) { echo "Error: " . $conn->error . "<br>"; return; }
  if ($res->num_rows > 0) {
    echo "<table border='1' cellpadding='5'><tr>";
    while ($f = $res->fetch_field()) echo "<th>{$f->name}</th>";
    echo "</tr>";
    while ($row = $res->fetch_assoc()) {
      echo "<tr>";
      foreach ($row as $v) echo "<td>$v</td>";
      echo "</tr>";
    }
    echo "</table>";
  } else echo "0 results found.<br>";
}

// 2. Reusable UPDATE / DELETE Runner
function runUpdate($conn, $title, $sql) {
  echo "<h4>$title</h4>";
  if ($conn->query($sql) === TRUE) {
    echo "Updated successfully. Rows affected: " . $conn->affected_rows
. "<br>";
  } else echo "Error: " . $conn->error . "<br>";
}

// 3. Extract Single Value (Total Net Worth, Count)
function showSingle($conn, $title, $sql, $colName) {
  $res = $conn->query($sql);
  if ($res && $row = $res->fetch_assoc()) {
    echo "<p><b>$title: </b>" . number_format($row[$colName], 2) . " BD
T</p>";
  }
} ?>
```

ZERO-CRASH EMERGENCY CHEAT SHEET (WHEN IT BREAKS)

- **TRAP 1:** Never put `COUNT(*)` or `AVG()` in `WHERE`! Aggregates MUST go in `HAVING` after `GROUP BY`.
- **TRAP 2:** Always run tasks in the **exact order listed**! Earlier updates change table rows for later queries.
- **TRAP 3:** Double check column spelling! Match exact casing from the question paper.
- **TRAP 4:** In XAMPP, open PHP via `http://localhost/web_final/...` (never double click file!).
- **TRAP 5:** Blank white screen in PHP? Missing `;` or closing brace `}`.
- **TRAP 6:** JS button does nothing? F12 → Console: check element ID and function name.

ALL 6 SQL QUERY ARCHETYPES (COVERING 100% OF PAST & FUTURE EXAMS) — SQL PATTERNS

```
-- ARCHETYPE 1: Category Summary & Aggregation with HAVING
SELECT category, COUNT(*) AS total_items, AVG(rating) AS avg_rating, SU
M(price) AS total_val
FROM records GROUP BY category HAVING AVG(rating) > 4.0 ORDER BY avg_ra
ting DESC;

-- ARCHETYPE 2: Conditional Filtered SELECT with OR / LIKE / BETWEEN
SELECT name, category, rating FROM records
WHERE rating > 4.5 OR name LIKE '%Beach%' ORDER BY rating DESC;

-- ARCHETYPE 3: Multi-Column Conditional UPDATE
UPDATE records SET status = 'Grace Period', price = 0.00
WHERE status = 'Overdue' AND stock < 7;

-- ARCHETYPE 4: Capped Increments & Percentage Discounts (+10%, -10%, +
5000 with CAP)
-- 4a. Percentage increase (+10%):
UPDATE records SET price = price * 1.10 WHERE price > 400;
-- 4b. Percentage discount (-10%):
UPDATE records SET price = price * 0.90 WHERE category = 'Language';
-- 4c. Fixed bonus with upper ceiling cap:
UPDATE records SET price = price + 5000 WHERE price > 50000 AND (price
+ 5000) ≤ 60000;

-- ARCHETYPE 5: Subquery & Category Average Comparison (CASE WHEN ... E
ND)
SELECT r1.name, r1.category, r1.price,
  CASE WHEN r1.price > (SELECT AVG(r2.price) FROM records r2 WHERE r2.c
ategory = r1.category)
       THEN 'Top Seller' ELSE 'Regular Seller' END AS seller_status
FROM records r1 ORDER BY r1.price DESC;

-- ARCHETYPE 6: Whole Table Single Value (Total Net Worth: SUM(price *
stock))
SELECT SUM(price * stock) AS total_net_worth FROM records;
```

EXAM WORDING → SQL QUERY MAPPING DICTIONARY

When Question Asks:	The Exact SQL Command to Write:
"Show / Display / Find / List"	`SELECT col1, col2 FROM table`
"For each category / department / per..."	`GROUP BY CategoryName` (must be in SELECT too)
"Total number of ... in each ..."	`SELECT col, COUNT(*) AS total FROM t GROUP BY col`
"Only include groups having more than N / avg > X"	`HAVING COUNT(*) > N` or `HAVING AVG(rating) > 4.4`
"Sorted largest first / highest to lowest"	`ORDER BY col_name DESC`
"Sorted smallest first / ascending"	`ORDER BY col_name ASC`
"Name contains word 'Beach' or 'Forest'"	`WHERE name LIKE '%Beach%'`
"Rating between 4.0 and 4.5 (inclusive)"	`WHERE rating BETWEEN 4.0 AND 4.5`
"If salary < 40,000 and rating != 'D', change to 'C'"	`UPDATE t SET rating='C' WHERE salary < 40000 AND rating <> 'D'`
"Add 5,000 bonus, only if resulting ≤ 60,000"	`UPDATE t SET sal = sal + 5000 WHERE sal > 50000 AND (sal + 5000) ≤ 60000`
"Add 10% bonus revenue / Increase by 10%"	`UPDATE t SET rev = rev * 1.10 WHERE rev > 70000`
"Reduce price by 10% / 10% discount"	`UPDATE t SET price = price * 0.90 WHERE category = 'Language'`
"Total estimated revenue: sum of fee × visitors"	`SELECT region, SUM(fee * visitors) AS rev FROM t GROUP BY region`
"Total net worth of entire bookshop"	`SELECT SUM(price * stock) AS total_worth FROM book_info`
"Label 'Top Seller' if above own category avg"	`CASE WHEN price > (SELECT AVG(price) FROM t t2 WHERE t2.cat = t.cat) THEN 'Top Seller' ELSE 'Regular' END`
"Change two columns at once"	`UPDATE t SET status='Grace Period', fee=0.00 WHERE status='Overdue' AND days < 7`